

On-Chain Groth16 Verifier Export Bug

Toolchain Investigation and Reproduction Strategy

Perry Kundert

2026-06-05

Contents

1	Symptom	1
2	Eliminated Causes	2
3	Working Reference	2
4	Hypothesis	2
5	Reproduction Strategy	2
6	Reproduction Commands	3
7	Key Unknown	3

1 Symptom

Freshly exported Solidity Groth16 verifiers fail on-chain with `PrecompileError` on the `bn256Pairing` precompile (EIP-197). The off-chain `snarkjs groth16 verify` passes. Old verifiers generated before the `nixpkgs` upgrade continue to work.

The failing gas value is the sentinel `0xF800000000000000000000000000000017` ($\sim 1.78e19$) — a forge-internal error code for precompile failure.

2 Eliminated Causes

Cause	Tested	Result
circom version	2.2.2 (old nixpkgs) vs 2.2.3 (26.05)	Both fail for fresh verifiers
snarkjs version	0.7.5 vs 0.7.6	Both fail
forge version	1.5.1 (old nixpkgs) vs 1.7.1 (26.05)	Both fail
solc version	0.8.28 (old nixpkgs) vs 0.8.33 (26.05)	Both fail
npm transitive deps	Locked, integrity hashes match	Eliminated
ptau files	Unchanged, deterministic	Eliminated
field element bounds	All values $< r$	Eliminated
circuit complexity	Flat identity _{membership} (no circom-lib)	Still fails

3 Working Reference

The verifier `src/IdentityMembershipVerifier.sol` from commit 7d2f686 ("Notes: wire the unified membership circuit") works on-chain. It was generated with:

- circom 2.2.2 + snarkjs 0.7.5 on node 20
- The standard `zkey export solidityverifier` command

4 Hypothesis

The most likely remaining cause: non-determinism in snarkjs zkey setup. The `groth16 setup` step reads the R1CS and ptau and produces a `zkey`. While the ptau is deterministic, the setup phase may incorporate a random seed that affects the verification key. The off-chain `snarkjs verify` uses the JSON VK (correct), but the exported Solidity verifier embeds VK constants that may not match.

5 Reproduction Strategy

1. Document the exact snarkjs incantations.
2. Run the same circuit through the pipeline twice.
3. Compare the exported Solidity verifier IC constants.
4. If they differ between runs, the setup is non-deterministic.

5. If they match, compare with the historical working verifier.

6 Reproduction Commands

```
# Compile circuit
circom circuits/identity_membership.circom --r1cs --wasm --output build/snark/idmem

# Generate witness
python scripts/snark/gen_idmem_witness.py > build/snark/idmem/input.json
snarkjs wtns calculate build/snark/idmem/identity_membership_js/identity_membership.wasm
    build/snark/idmem/input.json build/snark/idmem/witness.wtns

# Setup (use deterministic entropy)
snarkjs groth16 setup build/snark/idmem/identity_membership.r1cs \
    build/snark/ptau/pot15_final.ptau build/snark/idmem/zkey_0000.zkey
snarkjs zkey contribute build/snark/idmem/zkey_0000.zkey build/snark/idmem/zkey_0001.zkey \
    --name="test" -v -e="fixed-entropy"

# Prove + verify off-chain
snarkjs groth16 prove build/snark/idmem/zkey_0001.zkey \
    build/snark/idmem/witness.wtns build/snark/idmem/proof.json build/snark/idmem/public.json
snarkjs groth16 verify build/snark/idmem/verification_key.json \
    build/snark/idmem/public.json build/snark/idmem/proof.json

# Export Solidity verifier
snarkjs zkey export solidityverifier build/snark/idmem/zkey_0001.zkey \
    build/snark/idmem/Verifier.sol
```

7 Key Unknown

Even with fixed entropy (`zkey contribute -e "fixed-entropy"`), the `groth16 setup` step itself may introduce randomness. If so, the fix is to identify and control that entropy source.